
Problem Set 3: Big Data & Machine Learning

Juan Esteban Díaz Torres¹, Natalia Plata Ángel², Ángel David Ramírez Torres³ y Juan Pablo Grimaldos⁴

¹ 202020319 ² 202013152 ³ 202112704 ⁴ 202122627

[Repositorio.](#)

1. Introducción

Estimar correctamente precios de vivienda en el mercado inmobiliario es de suma importancia para diversos actores económicos. Para los compradores tener información precisa les permite tomar mejores decisiones con respecto a su activo más importante. En la mayoría de los casos la vivienda es el activo más valioso de una persona y simultáneamente su mayor fuente de deuda Bhatt.(2024). Tener una vivienda propia genera estabilidad, seguridad y es fundamental para el bienestar de cualquier ser humano como está consignado en la declaración universal de los derechos humanos. Para los constructores es indispensable conocer qué tipo de viviendas están demandando las personas y a que precio podrían venderlas en el mercado para decidir que tipos de proyectos ejecutar y cuáles son los riesgos asociados. Para las entidades financieras es de suma importancia conocer el precio de los inmuebles para conceder y diseñar correctamente créditos hipotecarios, así como para visualizar tendencias en el mercado Sharma et al.(2024).

En este contexto, los algoritmos de Machine Learning son una herramienta muy efectiva para aproximarse al problema. Limsombunchai, Gan Lee.(2004) encontraron que una red neural tiene un mejor desempeño predictivo que una regresión hedónica dentro y fuera de muestra debido a su capacidad para adaptarse a funciones más complejas. El Mouna et al.(2023) compararon el desempeño predictivo de una regresión lineal, Random Forest y Gradient Boosting para predecir precios de vivienda. Los autores encontraron que Gradient Boosting fue el modelo que logro el mejor desempeño. Asimismo, Sharman, Harsora Ogunleye.(2024) compararon el desempeño de una regresión lineal, Support Vector Machines, una Red Neuronal Multicapa y XGBoost para predecir precios de vivienda y encontraron que el modelo con la mayor capacidad predictiva fue XGBoost. Los autores resaltan que además del mejor desempeño XGBoost permite identificar la importancia de las variables.

Para aproximarnos a este problema contamos con datos de Properati sobre las características estructurales de 48,930 propiedades residenciales en la ciudad de Bogotá (área, número de baños, número de habitaciones, entre otras) en conjunto con datos provenientes de Datos Abiertos Bogotá y Open Street Map sobre las amenidades locales cercanas a los inmuebles (parques, hospitales, restaurantes, distancia al centro de negocios de la ciudad, entre otras). La base consolidada fue preprocesada principalmente para imputar valores faltantes de las características estructurales de los inmuebles a partir del título y la descripción de estos y para calcular la distancia de las propiedades a diversas amenidades. Utilizamos estos datos para entrenar algoritmos de Machine Learning sobre los que ahondaremos más adelante.

De los algoritmos entrenados el que mejor desempeño predictivo tuvo fue XGBoost con un MAE de alrededor de 187 millones de pesos seguido por un SuperLeaner que pondera las predicciones de un árbol de decisión y regresión lineal (MAE de 193 millones), Random Forest (MAE de 207 millones), una Red neuronal Multicapa (MAE de 218 millones) y finalmente un Elastic Net (MAE de 242 millones). Los modelos más flexibles y menos propensos a sobre ajustar los datos fueron los que obtuvieron mejores resultados. Esto evidencia que la relación entre los precios de las propiedades y sus características es compleja y requiere de modelos flexibles capaces de aproximarla sin sobre ajustar los datos. Nuestros resultados son consistentes con los antecedentes revisados.

2. Descripción de los datos

2.1. Fuente

Los datos utilizados en este ejercicio provienen de Properati, una plataforma digital de bienes raíces en América Latina que facilita la búsqueda, compra, venta y alquiler de propiedades. La base de datos contiene información sobre 48.930 casas y apartamentos ubicados en el territorio de Bogotá. Se dispone de información sobre el precio para 38.644 inmuebles, mientras que 10.286 presentan datos faltantes en esta variable. Estos inmuebles con información incompleta se concentran principalmente en la localidad de Chapinero. Los registros incluyen la descripción del apartamento, el título del anuncio, el precio solicitado, y otras características del inmueble, como el área, la cantidad de habitaciones, baños y otras variables relevantes.

Para complementar la información de Properati, se utilizaron dos fuentes principales de datos espaciales de la ciudad de Bogotá. La primera es Datos Abiertos Bogotá, una plataforma distrital que promueve el acceso a la información pública generada por las entidades del gobierno local. De esta fuente se obtuvieron variables como localidad, sector, estrato socioeconómico, avalúo catastral y comercial por manzana, niveles de criminalidad, presencia de restaurantes y museos, área residencial por manzana, indicadores de espacio público verde y espacio público efectivo, ubicación de Centros de Atención Inmediata (CAIs), estaciones de transporte público, recaudo predial, bibliotecas, alumbrado público y red de ciclorutas. La segunda fuente fue OpenStreetMap, un mapa colaborativo, gratuito y de código abierto, construido por una comunidad global de voluntarios. De allí se extrajo información sobre la ubicación de centros comerciales, hospitales, parques y tiendas.

Estos datos resultan fundamentales para el desarrollo del ejercicio, ya que, según el modelo de precios hedónicos de Rosen, el valor de un inmueble está determinado tanto por sus atributos estructurales como por las características y amenidades del entorno en el que se encuentra ubicado.

2.2. Procesamiento

2.2.1. Procesamiento de texto

$$\mathbf{X}_t = \left\{ \begin{array}{l} \text{Cocina americana, Cocina integral, Parqueadero de visitantes, Lavandería, Gimnasio, Balcón,} \\ \text{Seguridad, Walking closet, BBQ, Terraza, Depósito, Chimenea, Conjunto, Ascensor,} \\ \text{Patio, Dúplex, Piscina, Sauna, Jacuzzi, Altílo, Zona verde, Número de baños, Área,} \\ \text{Número de parqueaderos} \end{array} \right\}$$

En cuanto a las variables construidas a partir del texto se añadieron un total de 24 variables desde las variables “description” y “title”, de las cuales 3 son variables numéricas y 21 son variables dicotómicas. La lista $\mathbf{X}_t$ contiene los nombres de las variables construidas. Las tres variables numéricas son: área, número de baños y número de parqueaderos. El resto de las variables dicotómicas indican si esa característica hace parte de la vivienda.

El primer paso para la construcción de las variables fue combinar en una variable las dos variables de texto “description” y “title” de la base de datos original. Luego se buscó la frecuencia de cada palabra, par de palabras y trío de palabras, esto con la intención de buscar características compuestas por una, dos o tres palabras. Esto se realizó creando dos variables en las que estaban todas las posibles combinaciones en pares y tríos de la variable de texto construida anteriormente. Con estas variables se obtuvo la frecuencia de aparición de cada combinación y palabra individual, para ser revisada manualmente e identificada como coherente y relevante. Luego de identificar las características reportadas más comunes se revisó la codificación de manera manual, es decir, había características descritas de diferente – por lenguaje o por error – tanto como las palabras individuales como para los conjuntos de palabras. Luego de corregir por esta codificación se le asignó el valor de 1 a las viviendas que contaban

con características de si o no, para las variables numéricas se tuvo un tratamiento diferente. Para las tres variables numéricas se tuvo que identificar el número anterior a la característica referente, para luego sumar los números que estuvieran presentes más de una vez en la descripción. Por ejemplo, la variable de área tenía diferencias en su codificación, se reportaba como metros cuadrados, mts2, etc, y se reportaba primero el área del primer piso y luego la del segundo piso. Luego de obtener todas las variables se realizó la imputación de las variables numéricas, dado que en las variables dicótomas se dejó como cero la inexistencia de esa característica. Para la variable de número de parqueaderos se consideró que si la característica no estaba en la descripción entonces no se contaba con ella. Para las variables de área y número de baños primero se tomaron los valores de la base original como verídicos y se reemplazaron los valores faltantes de la base original por los valores obtenidos a partir de la descripción. Como segundo paso, se imputaron los valores faltantes del área por medio de la media de los datos agrupados en barrio, habitaciones, baños, estrato y tipo de propiedad, cambiando el barrio por la upz para los valores que no se imputaron con barrio – barrio y upz se importaron de la base de datos espaciales –. La variable de baños se imputó de la misma manera añadiendo primero el área individual y luego agrupada de 10 en 10. De las variables obtenidas se resalta la variable área y número de baños como unas de las más importantes dentro de la estimación, el área como la que recoge en mayor parte el precio de las viviendas. Dada la importancia del precio por metro cuadrado en el mercado inmobiliario una buena aproximación del área es relevante en la estimación.

2.2.2. Datos espaciales

Se utilizaron distintas metodologías para vincular los datos espaciales con la base de inmuebles de Properati. Para empezar, se calculó la distancia entre cada apartamento y diferentes tipos de lugares de interés (como colegios, estaciones de transporte, parques, etc.). Esto se basa en la idea de que la cercanía a ciertos lugares puede influir en el valor de un inmueble. Por ejemplo, una vivienda podría ser más costosa si se encuentra cerca de lugares frecuentados en la vida cotidiana, como colegios, lugares de trabajo o centros comerciales.

Para esto, se midió la distancia entre cada apartamento y todos los lugares de un mismo tipo (por ejemplo, cada colegio), y luego se guardó la distancia más corta. Es decir, para cada apartamento, se identificó cuál era el colegio más cercano, el parque más cercano, la estación más cercana, etc. En este ejercicio se trabajó con tres tipos de datos espaciales:

- Puntos, como las estaciones del SITP o los CAIs. En estos casos, se calculó la distancia euclidiana entre el apartamento y cada punto, y se guardó la distancia más corta.
- Polígonos, como parques o zonas residenciales. Como los polígonos tienen forma y tamaño, se simplificaron calculando su centroide. Luego, se midió la distancia entre el apartamento y ese centroide.
- Líneas, como las ciclorutas o rutas de transporte público. Estas líneas se dividieron en muchos puntos a lo largo de su recorrido, y se calculó la distancia entre el apartamento y todos esos puntos, conservando nuevamente la distancia mínima.

Por otra parte, en el caso de polígonos como localidad, UPZ, sector y manzana, no fue necesario calcular distancias. En estos casos, bastó con identificar en qué polígono se encuentra cada apartamento, a partir de sus coordenadas geográficas. Una vez asignado el apartamento a su correspondiente polígono, fue posible asociarle las características propias de ese territorio. De esta manera, se incorporaron variables como el número de luminarias por UPZ, el estrato socioeconómico por manzana y los indicadores de espacio público verde por localidad y UPZ.

Finalmente, se incluyó una medida de distancia entre cada apartamento y el centro de negocios de la ciudad, que se definió como el punto central entre las localidades de Chapinero, Teusaquillo, Santa

Fé y Los Mártires, que corresponden a las localidades con el mayor flujo de personas durante la hora pico que reportan desplazarse para ir a trabajar (Secretaría Distrital de Movilidad, 2019).

2.3. Estadísticas descriptivas

Las propiedades con el mayor precio por metro cuadrado están ubicadas principalmente en las localidades de Usaquén, Chapinero y Santa Fé al norte y noreste de la ciudad. En estas localidades también está la mayor concentración de propiedades de estratos 4, 5, y 6. Las propiedades con el menor precio por metro cuadrado están ubicadas principalmente en las localidades de Ciudad Bolívar, Usme, Tunjuelito, Kennedy, Bosa, Engativá y Fontibón al sur y al oeste de la ciudad. En estas localidades también está la mayor concentración de propiedades de estratos 0, 1 y 2 (Figuras 1 y 2). Esto indica que Bogotá es una ciudad segmentada en torno a las características económicas de las propiedades, donde las propiedades y en consecuencia la población se agrupa a partir de sus niveles de ingreso. Por otro lado, el valor de las propiedades también está influenciado por su cercanía al centro de negocios de la ciudad (CBD). Las propiedades que están a una distancia media del CBD tienen el mayor precio por metro cuadrado lo que implica que las personas valoran estar lo suficientemente cerca al CBD para no incurrir en altos costos de transporte, pero no quieren vivir exactamente en él (Figura 3). Con respecto a las características de las propiedades que están en la base de testeo las que están en la base de entrenamiento tienen en promedio: un área más grande, casi una habitación adicional, el mismo número de baños y parqueaderos, son de estrato 4 y las de testeo son de 5 y tienen en promedio la misma distancia hacia el parque, estación de Transmilenio, CAI y hospital más cercano. Es importante notar la presencia de valores extremos en las características estructurales de las propiedades, lo cual podría afectar el entrenamiento de los modelos (Tabla 1).

Figura 1. Precio por metro cuadrado

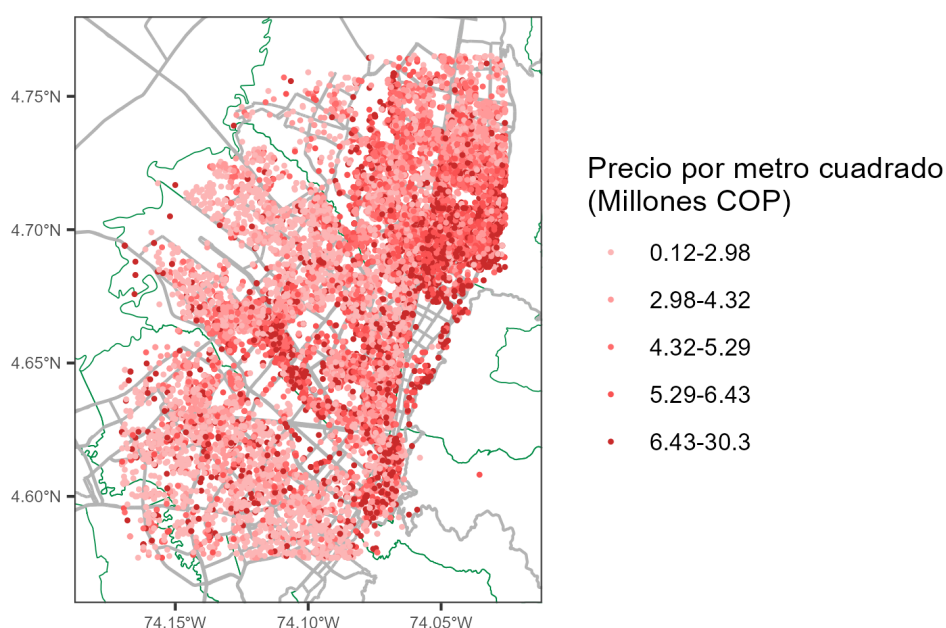


Figura 2. Estrato por manzana

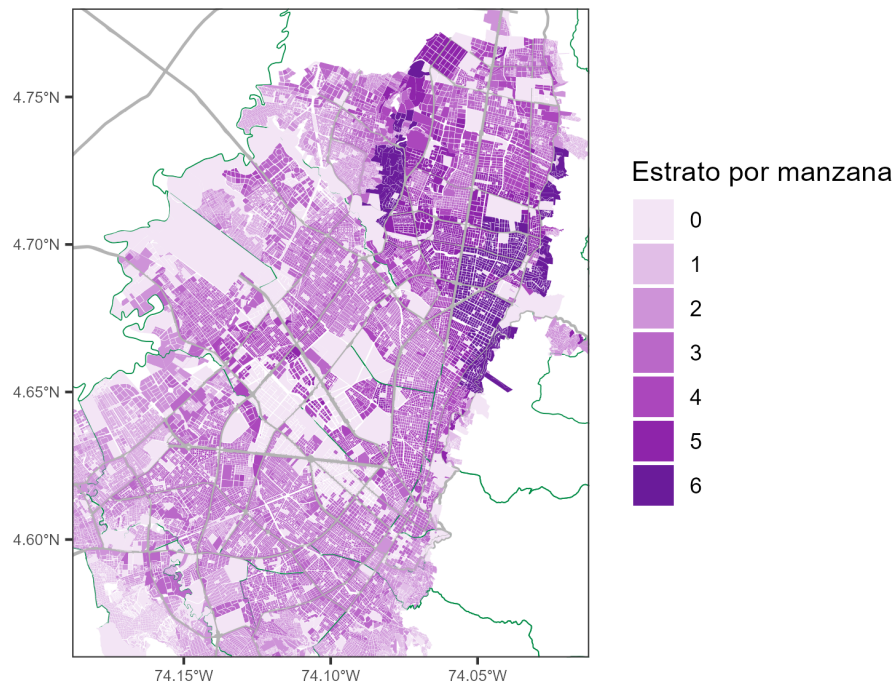


Figura 3. Precio por metro cuadrado por decil de distancia al centro de negocios

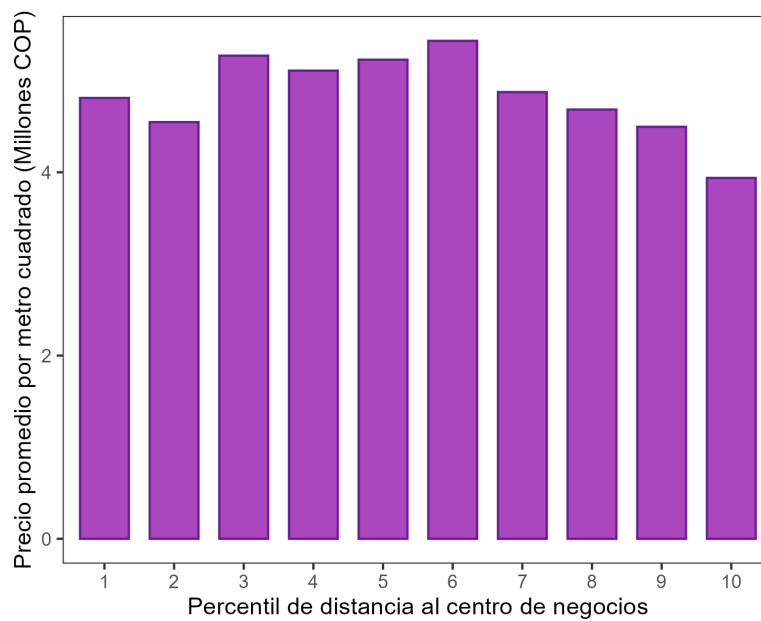


Tabla 1. Estadísticas Descriptivas: Bases de Entrenamiento y Testeo

Base de Entrenamiento					
Estadística	Mean	St. Dev.	Min	Max	Median
Habitaciones	3.145	1.535	0	11	3
Baños	2.858	1.093	1	16	3
Área	303.980	566.106	54.500	2,500.000	123.000
Parqueaderos	0.888	1.077	0	8	0
Estrato	4.239	1.574	0	6	4
Dist_Parque	0.158	0.097	0.001	3.061	0.138
Dist_TM	1.155	0.784	0.001	5.453	0.965
Dist_CAI	0.767	0.375	0.002	2.957	0.737
Dist_Hospital	0.923	0.535	0.010	3.566	0.811
Base de Testeo					
Estadística	Mean	St. Dev.	Min	Max	Median
Habitaciones	2.381	0.961	0	11	2
Baños	2.693	0.946	1	10	3
Área	251.272	491.980	54.500	2,500.000	130.000
Parqueaderos	1.046	1.086	0	8	1
Estrato	5.237	1.402	0	6	6
Dist_Parque	0.160	0.098	0.005	1.730	0.145
Dist_TM	0.994	0.524	0.002	3.845	0.973
Dist_CAI	0.659	0.401	0.008	2.732	0.572
Dist_Hospital	0.864	0.402	0.011	3.421	0.845

Nota: Dist_ indica distancia en kilómetros y Área se mide en metros cuadrados.

3. Modelos de clasificación

3.1. Elastic Net

El primer intento de predicción se realizó utilizando el método de regularización Elastic Net. Este método combina las ventajas de Lasso y Ridge: por un lado, Lasso reduce la varianza del modelo al llevar algunos coeficientes exactamente a cero, lo que favorece la selección de variables; por otro lado, Ridge penaliza los coeficientes grandes para evitar el sobreajuste, sin eliminarlos completamente. El modelo Elastic Net tiene dos hiperparámetros principales: α , que define si el modelo se comporta más como Lasso o Ridge, y λ , que regula la magnitud de la penalización.

En esta ocasión, la elección de hiperparámetros se realizó por medio del método de validación cruzada espacial en 5 folds. Además, no se definió una grilla para la búsqueda de los hiperparámetros del modelo, permitiendo que el software seleccione las combinaciones de hiperparámetros a explorar durante el proceso de entrenamiento del modelo. En este modelo se utilizaron la mayoría de las variables disponibles en la base de datos procesada. Además, las variables numéricas fueron normalizadas para evitar que aquellas variables con coeficientes de mayor magnitud reciban una penalización más grande. En este entrenamiento se manejó el logaritmo natural del precio como variable objetivo y luego se transformaron los valores predichos a pesos, esto con el fin de reducir el impacto de los valores atípicos sobre los resultados de la predicción. que el resultado del entrenamiento de este modelo se puede ver en la tabla 2.

Tabla 2. Mejores combinaciones de hiperparámetros en Elastic Net

Penalty	Mixture	MAE (mean)
0.00000000129	0.789	0.251
0.00000278	0.894	0.251
0.000000215	0.472	0.251
0.0000000001	0.367	0.251
0.0000359	0.156	0.251

La tabla muestra que el mejor modelo obtuvo un Error Absoluto Medio (MAE) de validación cruzada de 0.251 y sus hiperparámetros fueron $\alpha = 0.00000000129$ y $\lambda = 0.789$. Al cargar este modelo a Kaggle, el MAE de prueba de este modelo fue de 232.5 millones de pesos. Este modelo resultó útil para la selección de variables en algoritmos más complejos y computacionalmente costosos, como el Bosque Aleatorio.

3.2. Bosque Aleatorio

Un bosque aleatorio (random forest) es un modelo de aprendizaje automático basado en un conjunto de árboles de decisión que trabajan en conjunto para generar predicciones más precisas y robustas. Cada árbol se entrena sobre una muestra aleatoria de los datos obtenida mediante bootstrap, y en cada división considera solo un subconjunto aleatorio de variables, lo que contribuye a reducir el riesgo de sobreajuste. En el contexto de predicción de precios de vivienda, este enfoque permite capturar relaciones complejas y no lineales entre múltiples características del inmueble, o el entorno en el que está ubicado, y su valor de mercado.

Las variables incluidas en este modelo fueron aquellas que obtuvieron coeficientes distintos de cero en el modelo de Elastic Net, lo cual permite excluir predictores con baja capacidad explicativa y, al mismo tiempo, reducir el costo computacional del entrenamiento. Adicionalmente, se excluyó la variable de localidad, ya que al convertirla en variables ficticias (dummies), se incrementaba significativamente la dimensión de la matriz de predictores, encareciendo el proceso computacional. Nuevamente se utilizó el logaritmo natural del precio como variable objetivo.

En el entrenamiento de este modelo se probaron distintas grillas para la selección de hiperparámetros en este modelo. La búsqueda del número de predictores a seleccionar para generar las particiones de los nodos de cada árbol se enfocó en un rango entre 5 y 50 predictores (de un total de 59 predictores). Por otra parte, la búsqueda del número mínimo de observaciones por nodo se realizó en un rango entre 2 y 20 observaciones. Por último, el entrenamiento de este modelo también se realizó utilizando validación cruzada espacial en 5 folds.

Como resultado de este proceso se enviaron tres Bosques Aleatorios a la competencia en Kaggle. Uno de ellos tenía tan solo 10 árboles y fue entrenado en una grilla con 25 combinaciones de los hiperparámetros. Este modelo obtuvo un error de prueba de 220.2 millones de pesos. El siguiente modelo fue entrenado con la misma grilla pero con 1000 árboles. Este modelo obtuvo un error de 208.7 millones de pesos, lo que muestra una mejora de 11.7 millones respecto al bosque de 10 árboles. Por último, se entrenó un modelo de 50 árboles, ajustando la grilla utilizada para hallar los hiperparámetros alrededor de los valores en los que se encontró el menor error absoluto medio de validación cruzada en los anteriores modelos. Por lo tanto, se redujo el rango de búsqueda del número de predictores para las particiones de los nodos a un rango entre 10 y 45 predictores. Además, se aumentó el tamaño de la grilla a 64 combinaciones de los hiperparámetros. Este modelo obtuvo un error de predicción de 211.4 millones de pesos, lo cual está más cerca del error del bosque con 1000 árboles que el de 10. Este ejercicio muestra la importancia de seleccionar una grilla adecuadamente antes de estimar modelos con mayor cantidad de árboles, con el fin de mejorar el error de predicción de estos modelos.

Tabla 3. Resultados de Random Forest

# Árboles	Predictores	Min. Obs.	MAE (Cross)	MAE (Kaggle)
1000	20	2	0.219	208,711,626.67
50	27	18	0.221	211,363,188.93
10	35	20	0.230	220,200,000.00

3.3. Boosting

3.3.1. Descripción del algoritmo

Con el propósito de estimar una predicción robusta del precio de las viviendas en el conjunto de validación, se utilizó también el método de estimación de Boosting como herramienta. Este algoritmo se basa en la combinación de un gran número de árboles de decisión $\hat{f}^1, \dots, \hat{f}^B$. El Boosting ajusta árboles de decisión a los residuales de los modelos estimados como mecanismo de aprendizaje. Se procede entonces a añadir el árbol recién creado a la función ajustada con el propósito de actualizar los residuales, de modo que:

$$\begin{aligned}\hat{f}(x) &\leftarrow \hat{f}(x) + \lambda \hat{f}^b(x). \\ r_i &\leftarrow r_i - \lambda \hat{f}^b(x_i).\end{aligned}$$

Cada uno de los árboles nuevos (ajustados a los residuales) es de menor tamaño (nodos terminales determinados por d) que el árbol original. Al ajustar pequeños árboles a los residuales, se mejora la función con la que se estima la predicción en áreas en donde es deficiente. El parámetro de encogimiento λ reduce la velocidad de aprendizaje al permitir b iteraciones de los árboles ajustados con los cuáles se actualizan los residuales. Al final del algoritmo, la predicción obtenida por boosting estaría dada por:

$$\hat{f}(x) = \sum_{b=1}^B \lambda \hat{f}^b(x).$$

3.3.2. Hiperparámetros y XGBoost

De acuerdo con la descripción anterior, generalmente hay tres hiperparámetros claves en el algoritmo de Boosting:

1). El número de árboles construídos para ajustar los residuales, b . Este parámetro es esencial en el proceso de Boosting, pues representa el número de veces que se realiza el proceso de aprendizaje mencionado anteriormente. Un número muy alto de b puede sobreajustar sobre los datos de entrenamiento; por ende, es deseable encontrar este parámetro a través de validación cruzada.

2). El factor de encogimiento λ . También entendido como una "tasa de aprendizaje", controla la tasa a la cuál se aprende de los residuales. Un valor común para este parámetro es 0.01, el cuál utilizaremos con frecuencia en la construcción de los modelos.

3). La complejidad del árbol, controlada por d . Este parámetro determina el número de particiones en cada árbol. También entendido como la profundidad de las interacciones del árbol. Usualmente se fija entre valores de 4 y 8.

A su vez, estos parámetros construyen "Weak Learners". No obstante, para estimar las predicciones, se utiliza XGBoost; una implementación optimizada del algoritmo de "Gradient Boosting". Este algoritmo pone un mayor énfasis en la minimización de la función de pérdidas que en exclusivamente el ajuste de pesos. Concretamente, se busca minimizar la función de pérdida de la predicción anterior mas el nuevo árbol a través de una aproximación de gradiente descendiente. XGBoost está diseñado para optimizar el rendimiento del cálculo de los pseudo-residuales, penalizando la complejidad del modelo (número de bifurcaciones y cuán lejos la predicción de la hoja está de ceros). Esto tiene implicaciones sobre el parámetro de profundidad del árbol, pues favorece a modelos menos ajustados pero con potencial mejor poder explicativo sobre los datos de validación. Esta aproximación asume los siguientes hiperparámetros adicionales:

4). La reducción en la función de pérdida mínima requerida para realizar una partición adicional en un nodo del árbol, γ . Entre más grande sea γ , más conservador será el algoritmo en generar nodos adicionales. Tiene rango $[0, \infty]$, pero se utilizarán valores $(0, 1)$.

5). El mínimo número de observaciones por nodo, z . si la partición del árbol resulta en un nodo en donde el total de observaciones sea menor a z , el algoritmo no hará esa partición.

6). La proporción de variables utilizadas para construir el árbol ajustado a los residuales, ϕ . Como en los Bosques Aleatorios, se toma una submuestra de las variables disponibles para aprovechar el carácter aditivo del algoritmo para encontrar información no explicada por el árbol anterior, e incorporarla al modelo.

7). La proporción de datos de entrenamiento utilizados para entrenar el algoritmo, κ . Este parámetro previene el sobreajuste. El remuestreo ocurre en cada iteración del Boosting.

3.3.3. XGBoost 1

El primer modelo entrenado a través del algoritmo previamente descrito utilizó como estrategia de validación una validación cruzada tradicional en 5 folds del conjunto de entrenamiento. La validación cruzada iteró sobre los siguientes valores de cada hiperparámetro: $b \in \{250, 500\}$, $\lambda \in \{0.1, 0.01\}$, $d \in \{1, 5\}$, $\gamma \in \{0, 1\}$, $z \in \{10, 25\}$, $\phi \in \{0.4, 0.7, 1\}$, $\kappa = 0.4$. El propósito de la fijación de estos hiperparámetros, bajo la luz de la discusión anterior, fue procurar por una estimación con baja varianza y por ende poco sobreajuste a los datos de entrenamiento. Se estima sobre una proporción reducida de la información de entrenamiento. Más aún, dada la dimensión de los datos, se buscó una estimación robusta y compleja que sería, a su vez, naturalmente penalizada por el algoritmo (dada por γ) con el fin de encontrar una profundidad óptima, a modo de inducción hacia atrás. Las bajas tasas de aprendizaje fueron contrarrestadas con un alto número de iteraciones. Las variables utilizadas para entrenar el modelo comprenden todas las variables de la base de datos a excepción de las variables clasificadoras de unidad de observación geográfica, como latitudes o longitudes, UPZs, barrios, localidades, manzanas, entre otras. Esto es porque en el conjunto de validación no existen algunos de los niveles observados en estas variables para el conjunto de entrenamiento. La lista completa de variables utilizadas está dada por el vector X_b .

$$\mathbf{X}_b = \left\{ \begin{array}{l} \text{Habitaciones, Tipo de Propiedad, Distancia a Biblioteca, Distancia a Colegio, Distancia a Museo,} \\ \text{Distancia a Parque, Distancia a Centro Comercial, Distancia a Transporte P blico, Estrato,} \\ \text{Aval o Catastral, Distancia a SITP, Homicidios, N mero de Lesiones, Hurtos a Personas,} \\ \text{Hurtos de Autos, Hurtos a Viviendas, Hurtos de Bicicletas, Hurtos de Motos,} \\ \text{Hurtos a Comercios, Hurtos de Celulares, Delitos Sexuales, Violencia Intrafamiliar,} \\ \text{Restaurantes Cercanos, Distancia a Cicloruta, Distancia al Centro, Distancia al CAI,} \\ \text{Distancia a Hospital, Distancia a Gimnasio, Distancia a Tienda, Distancia a Farmacia,} \\ \text{Tiene BBQ, Tiene Terraza, Tiene Dep sito, Tiene Chimenea, Cocina Americana,} \\ \text{Tiene Conjunto Residencial, Tiene Ascensor, Tiene Patio, Es D plex,} \\ \text{Tiene Piscina, Tiene Sauna, Tiene Jacuzzi, Tiene Altillo, Tiene Zona Verde,} \\ \text{N mero de Ba os,  rea, N mero de Parqueaderos, Aval o Comercial} \end{array} \right\}$$

Los hiperpar metros  ptimos seleccionados mediante validaci n cruzada para el modelo XGBoost fueron los siguientes: se construyeron $b = 500$  rboles con una profundidad m xima de $d = 5$, una tasa de aprendizaje de $\lambda = 0.1$, y una reducci n m nima en la funci n de p rdida para permitir una bifurcaci n de $\gamma = 1$. Adem s, se utiliz  el 100% de las variables disponibles en cada iteraci n ($\phi = 1$), un n mero m nimo de observaciones por nodo de $z = 10$, y una proporci n de datos de entrenamiento usada por iteraci n de $\kappa = 0.4$. La predicci n con estos par metros tuvo un $MAE = 187$ en millones COP. Esta fue la segunda mejor predicci n en t rminos relativos.

3.3.4. XGBoost 2

El segundo modelo entrenado a trav s de este algoritmo fue casi id ntico al primero: misma estrategia de validaci n, mismas variables (X_b), y casi todos los hiperpar metros fijados de la misma forma. El  nico cambio realizado fue fijar $b = 1000$, con el fin investigativo de observar si la tasa de aprendizaje era adecuada para maximizar el ajuste residual. Los par metros  ptimos no cambiaron a diferencia de b que ahora estaba fijo. La predicci n con esta aproximaci n tuvo un $MAE = 192$ en millones COP, lo cu l sugiere que un aumento en las iteraciones puede sobreajustar el modelo anterior y empeorar el ajuste predictivo del modelo a los datos de validaci n. Esta fue la tercera mejor predicci n en t rminos relativos.

3.3.5. XGBoost 3

El tercer modelo entrenado a trav s de este algoritmo utiliz  una estrategia de validaci n basada en validaci n cruzada espacial en 5 folds. En vista de la posible autocorrelaci n espacial, se quiere minimizar el efecto de "contaminaci n" de observaciones adyacentes geogr ficamente a los datos de entrenamiento. Esta metodolog a se enfoc  en la separaci n de las observaciones con una grilla basada en la longitud y la latitud de cada observaci n, fijando el sistema de referenciaci n de coordenadas EPSG 4326. Adicionalmente, para este proceso, se normalizaron las variables y se realizaron interacciones entre las variables de Tipo de Propiedad y Distancia a Parques, y Tipo de Propiedad y el  rea Total del inmueble para capturar efectos heterog neos. Se utilizaron todas las variables en X_b a excepci n de la variable de Distancia al Centro, pues no fue compatible con el algoritmo de normalizaci n. La validaci n iter  sobre los siguientes valores de los hiperpar metros: $b \in \{250, 500, 1000\}$, $\lambda \in \{0.01, 0.1\}$, $d \in \{3, 6, 10\}$, $\gamma \in \{0, 1, 10\}$, $\kappa \in \{0.4, 0.75, 1\}$, $\phi \in \{7/p, 25/p\}$, donde p representa el n mero total de predictores disponibles. Los hiperpar metros  ptimos obtenidos despu s de la validaci n fueron: $b = 1000$, $\lambda = 0.01$, $d = 6$, $\gamma = 0$, $z = 25$, $\phi = 1$, $\kappa = 1$. La predicci n con estos par metros tuvo un $MAE = 187$ en millones COP. Esta fue la mejor predicci n. La discusi n sobre el tipo de validaci n (tradicional vs. espacial) y las variables utilizadas se reserv  para la secci n en la que se discute a mayor profundidad el mejor modelo.

3.4. Red Neuronal

Se entren  una red neuronal con m ltiples capas para predecir el precio de los apartamentos en Chapinero. Durante el proceso de entrenamiento se implementaron tres arquitecturas con distintos

niveles de profundidad: Bajo (10 nodos y una sola capa oculta), Medio (128 nodos y 5 capas ocultas cada una con menos nodos que sus capas antecesoras) y Alto (256 nodos y 6 capas ocultas cada una con menos nodos que sus capas antecesoras). Para todas las arquitecturas se incluyó para todas las capas una capa que apaga aleatoriamente el 30% de los nodos para evitar sobre ajustar los datos. Después de revisar las arquitecturas vistas en clase para tener un punto de partida escogimos estas tres arquitecturas para ver cómo se comportaba el error fuera de muestra a medida que iba aumentando la profundidad del modelo.

De las tres arquitecturas ya mencionadas la que obtuvo mejores resultados fue la arquitectura de profundidad media (MAE de 218 millones) seguida por la baja (MAE 374 millones) y el alta (MAE 833 millones) respectivamente. De esto podemos inferir que la red de profundidad media logro el mejor balance entre aprender la función y no sobre ajustar los datos. Los tres modelos fueron entrenados con la misma base de datos que tenía 137 variables sobre las características estructurales de las viviendas (área, número de baños y habitaciones, estrato y comodidades del edificio como sauna, bbq piscina) y de su cercanía a diversas amenidades (parques, hospitales, restaurantes, arbolado público y al centro de negocios de la ciudad CBD, entre otras). El gran número de variables se debe al “one hot encoding” realizado para convertir las variables categorías en numéricas para poder estimar el modelo. Además, se re-escalo la variable de respuesta dividiéndola por un millón para estabilizar la red y mejorar la predicción.

Las redes neuronales entrenadas tuvieron en un peor desempeño que random forest, XGBoost y el Superlearner descritos en las otras secciones. Consideramos que este resultado ocurrió porque la flexibilidad de las redes genero un sobre ajuste de los datos cuando buscamos reducir más el error. Problema que no padecieron en la misma medida los demás modelos.

3.5. *Super Learner*

El modelo de Super Learner se realizó con dos conjuntos de algoritmos, el primero con regresión lineal, CART's y Elastic Net, al segundo conjunto se le agregó Random Forest. Se realizaron 3 estimaciones, una con cada grupo de modelos y otra con el primer modelo estimando el logaritmo natural de precio con un total de 54 variables independientes. La estimación con mejor ajuste del MAE fue el modelo con el segundo grupo de algoritmos, seguido del primer grupo con el precio sin alterar y por último el primer grupo con el logaritmo natural del precio.

El mejor desempeño del modelo de Super Learner que incluye Random Forest se explica por su capacidad superior para capturar relaciones no lineales complejas entre las variables predictoras y el precio de las viviendas. Este modelo obtuvo un error absoluto medio (MAE) de aproximadamente 193 millones, mientras que el Super Learner sin Random Forest alcanzó un MAE mucho mayor, de 233 millones. La diferencia en estos valores indica una mejora significativa en la precisión del modelo cuando se incorpora Random Forest al conjunto de algoritmos base.

El primer modelo muestra una limitación clara. La regresión lineal, aunque útil para identificar tendencias globales, parte de la suposición de relaciones lineales entre las variables independientes y la dependiente. CART's, por su parte, puede manejar cierta no linealidad, pero al ser un único árbol, tiende a ser inestable y propenso al sobreajuste. Elastic Net, al recibir un peso de 0 en el Super Learner, no contribuyó a mejorar la predicción, lo que indica que su desempeño fue inferior incluso en comparación con los otros dos modelos base.

En contraste, al añadir Random Forest en el segundo modelo, el conjunto mejora con un algoritmo que reduce la varianza de los árboles individuales mediante el promedio de múltiples árboles entrenados en subconjuntos aleatorios de los datos. Esta estructura le permite detectar interacciones sutiles y no lineales entre las variables, características comunes en el problema de predicción del precio

de viviendas, donde influyen múltiples factores como el área, la ubicación, entre otros.

4. Modelo final

Figura 4. Validación cruzada espacial con bloques definidos por coordenadas geográficas (EPSG:4326)

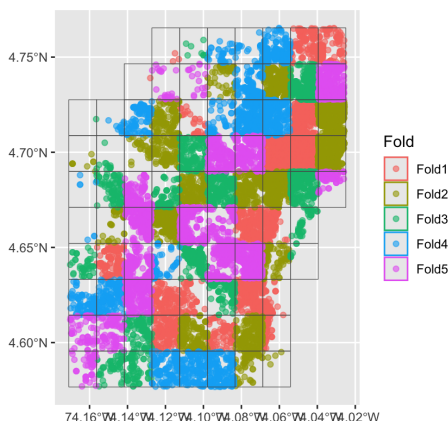


Tabla 4. Importancia relativa de las variables predictoras en el modelo XGBoost 3

	Variable	Importancia
1	Área	100.000
2	Baños	74.192
3	Estrato	33.334
4	Parqueaderos	30.502
5	Habitaciones	16.267
6	Distancia museo	8.787
7	Distancia colegio	7.904
8	Distancia Transmilenio	7.168
9	Distancia gimnasio	7.109
10	Tipo de propiedad: Casa	7.069

El modelo con la métrica de desempeño más sobresaliente fue la tercera iteración del modelo construido usando XGBoost (XGBoost 3). Los parámetros óptimos de este modelo fueron: $b = 1000$, $\lambda = 0.01$, $d = 6$, $\gamma = 0$, $z = 25$, $\phi = 1$, $\kappa = 1$. Este modelo utilizó como estrategia de validación una validación cruzada espacial de 5 folds separados en grillas de acuerdo con la longitud y latitud de las observaciones (ver Figura 4). A través de la opción "VarImp" del paquete caret en R, fue posible obtener una medida relativa de la importancia de las variables utilizadas en este modelo (ver Tabla 1). La predicción usando este modelo obtuvo un $MAE = 187$ en millones COP.

En primer lugar, se observa que, relativo a los demás modelos entrenados utilizando XGBoost, este fue el único que utilizó validación cruzada espacial. A diferencia de las otras iteraciones, el uso de validación cruzada espacial permitió reducir la contaminación por dependencia de las observaciones en el margen de los conjuntos de entrenamiento y de validación, y así producir generalizaciones con menor varianza predictiva y por consiguiente menor autocorrelación espacial. A pesar de que la mejora en el MAE fue marginal (relativo al segundo mejor modelo, XGBoost 1), se demuestra que esta aproximación espacial a la estrategia de validación sí tuvo un impacto positivo en la capacidad predictiva del modelo. La segmentación por longitud y latitud tuvo un alto nivel de granularidad para identificar observaciones dependientes geográficamente, potencialmente en mayor medida que una segmentación por barrio, localidad, o UPZ.

En segundo lugar, a través de este enfoque de validación, se encontraron hiperparámetros coherentes a la luz de la teoría vista en clase y las características del algoritmo de XGBoost y el Gradient Boosting. En particular, este modelo se caracterizó por tener un aprendizaje lento, contrarrestado por un gran número de iteraciones con el fin de prevenir el sobreajuste. Más aún, aunque la profundidad de cada árbol haya sido mayor que en los demás XGBoost y los requisitos para la reducción mínima en la función de pérdida para realizar particiones fueron nulos, se procuró por la limitación de la segmentación a través de un $z = 25$. A la misma vez, se utilizaron todas las variables y observaciones disponibles para ajustar el modelo. Aunque esta aproximación haya sido en su mayoría flexible, se mantuvieron restricciones principalmente sobre la división de nodos para evitar el sobreajuste.

En tercer lugar, como observado en la Tabla 4, las 10 variables más importantes para entrenar el modelo fueron obtenidas tanto a través del procesamiento de texto como del procesamiento de datos geográficos. Las cinco variables más importantes; Área, Número de Baños, Estrato, Número de Parqueaderos, y Número de Habitaciones fueron obtenidas a través de expresiones regulares. La importancia de estas variables no sólo es indicativa de su alta frecuencia de aparición en las entradas de texto de Properati, pero también de su relevancia para el avalúo de una vivienda. El modelo hubiera tenido potencialmente una mejor predicción si la variable de Distancia al Centro hubiera sido compatible con el proceso de normalización.

5. Conclusión

La pregunta que motivó el desarrollo de este análisis fue clara: ¿cuál es el mejor modelo para predecir el precio de las viviendas, y por qué? En la búsqueda por responderla, se entrenaron múltiples algoritmos con distintos niveles de complejidad y estrategias de validación, incluyendo Elastic Net, Random Forest, redes neuronales, distintos esquemas de XGBoost y el algoritmo de ensamble Super Learner. La evaluación de los modelos se realizó con base en el Error Absoluto Medio (MAE).

Entre todos los modelos entrenados, el mejor desempeño predictivo fue alcanzado por XGBoost 3, con un MAE de 187 millones de COP. Este modelo destacó no solo por sus hiperparámetros optimizados, sino especialmente por el uso de una validación cruzada espacial, que evitó el sobreajuste en zonas geográficas similares entre entrenamiento y validación. Su configuración combinó una alta capacidad de aprendizaje con mecanismos de regularización efectivos. Además, el modelo capturó relaciones complejas gracias a la inclusión de interacciones específicas y a una cuidadosa selección de variables. Las más influyentes —como área, número de baños, estrato y parqueaderos— reflejan atributos estructurales relevantes en el mercado inmobiliario, validados empíricamente como determinantes del valor de la propiedad.

El segundo mejor modelo fue XGBoost 1, seguido por Random Forest y Super Learner con Random Forest, todos con desempeños cercanos al óptimo. En particular, el Super Learner que incluyó Random Forest superó al que no lo hizo (MAE de 189.8 millones frente a 235.2 millones), evidenciando cómo la combinación de modelos base robustos y no lineales —como los bosques aleatorios— puede mejorar la precisión del conjunto. En contraste, el modelo sin Random Forest mostró un peso nulo para Elastic Net, indicando su escasa relevancia en el ensamblaje, y un sesgo hacia la regresión lineal y los CARTs, modelos más limitados en capturar interacciones y no linealidades complejas.

Respecto al modelo Elastic Net, si bien no resultó competitivo en términos de error de predicción, su utilidad radicó en la selección inicial de variables, permitiendo reducir la dimensionalidad y facilitar el entrenamiento de modelos más exigentes computacionalmente. Las redes neuronales, por su parte, ofrecieron resultados mixtos: su arquitectura de profundidad media logró un desempeño razonable (MAE de 218 millones), pero tanto las versiones más simples como las más profundas sufrieron de subajuste y sobreajuste, respectivamente. Esto confirma que, en contextos como este, el equilibrio entre complejidad y regularización es crítico, y que una alta flexibilidad no garantiza una mejor predicción.

References

1. Bhatt, G. (2024, December 4). Housing's Unique Role in Lives and Economies Demands Greater Understanding. *IMF Blog*. Recuper-

-
- ado de : <https://www.imf.org/en/Blogs/Articles/2024/12/04/housings-unique-role-in-lives-and-economies-demands-greater-understanding>
2. el Mouna, L., Silkan, H., Haynf, Y., Nann, M. F., & Tekouabou, S. C. K. (2023). A Comparative Study of Urban House Price Prediction using Machine Learning Algorithms. *E3S Web of Conferences*, 418. <https://doi.org/10.1051/e3sconf/202341803001>
 3. Limsombunchai, V., Gan, C., & Lee, M. (2004). House Price Prediction: Hedonic Price Model vs. Artificial Neural Network. *American Journal of Applied Sciences*, 1(3), 193–201. <https://doi.org/10.3844/ajassp.2004.193.201>
 4. Secretaría Distrital de Movilidad. (2019). *Resultados de la Encuesta de Movilidad de Bogotá y municipios vecinos*. Observatorio de Movilidad Bogotá. Recuperado de: <https://observatorio.movilidadbogota.gov.co/sites/observatorio.movilidadbogota.gov.co/files/2024-06-02/encuesta/Cartilla%20resultados%20Encuesta%20de%20Movilidad%202019.pdf>
 5. Sharma, H., Harsora, H., & Ogunleye, B. (2024). An Optimal House Price Prediction Algorithm: XGBoost. *Analytics*, 3(1), 30–45. <https://doi.org/10.3390/analytics3010003>